Other ensemble methods

29 October 2025 20:11

"Gradient-boosted decision trees build a team of weak learners *sequentially*. Each new tree tries to fix what the team got wrong so far. Think of it as a debate team: each new speaker addresses the mistakes or weak points left by earlier speakers."

Core mechanics

1. Start with a baseline prediction (e.g., mean label or prior probabilities).
2. Compute residuals (errors) — these are the targets for the next tree.
3. Fit a shallow tree to residuals; add it to the ensemble scaled by a **learning rate**.
4. Repeat for n_estimators rounds.

Intuition/analogy:

- Residuals = teacher's pointer to where the model needs to improve.
- Learning rate = microphone volume for each new speaker — lower means many quiet corrections, which tends to generalize better.
- Trees are weak learners — shallow, inexpensive specialists.

Important knobs (what they do in plain English)

- n_estimators: number of correcting steps (trees).
- learning_rate: how much each correction counts.
- max_depth/max_leaf_nodes: complexity of each correction.
- subsample / colsample_bytree: randomness to reduce overfitting.

Pitfall to call out live:

- Too many trees with high learning rate → overfit. Too few trees or too small learning rate → underfit or excessive training time.

Visual to describe on whiteboard/slides:

- Show baseline line, then add small step-wise corrections stacking on top (sketch residual-correction arrows).

optional

XGBoost = GBDT re-engineered. Same idea, but with engineering that makes it faster, more robust, and easier to regularize.

1. The Objective Function

For each boosting round, XGBoost minimizes:

$$\text{Obj} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Where:

- l = loss function (e.g., log loss, squared error)
- $\Omega(f_k)$ = regularization term for tree k

Now, **XGBoost's tree regularization** term is:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2 + \alpha \sum_j |w_j|$$

Where:

- T = number of leaves (penalizes larger trees)
- w_j = leaf weight (the output score at each leaf)
- λ = L2 regularization term
- α = L1 regularization term
- γ = cost per leaf (encourages pruning)

2. Intuitive meaning

- **L2 (λ)** = "rubber band" penalty — pulls large leaf weights toward zero smoothly.
  → Encourages smaller, more stable leaf outputs.
  → Makes model less sensitive to noise.

- **L1 (α)** = "lasso" penalty — can shrink some leaf weights *exactly to zero*.
  → This prunes unnecessary leaves altogether.
  → Produces sparser, simpler trees.

- **γ (gamma)** = "complexity tax" — adds cost per additional leaf.
  → Model only adds a leaf if it improves loss enough to justify that cost.

3. Analogy

Imagine you're designing a city of roads (the decision tree).

- λ (L2) = smooths all roads, discouraging sharp or extreme paths.
- α (L1) = removes unnecessary side roads entirely.
- γ (gamma) = charges you rent per road you build, so you only add roads that truly improve traffic flow.

Together they make trees smaller, faster, and generalize better.

EFB in Light GBMs

Problem:

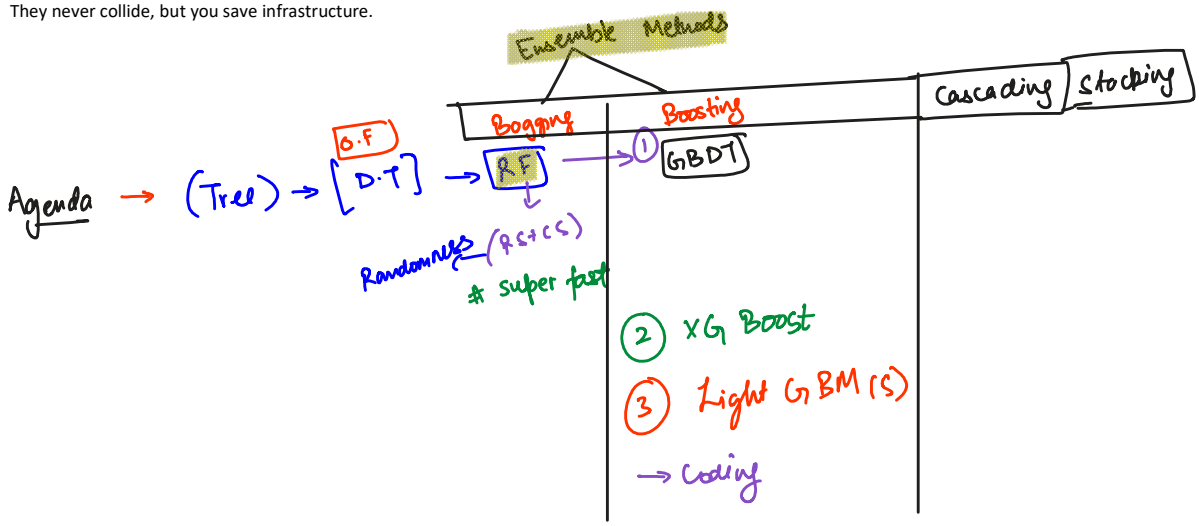
High-dimensional data (especially after one-hot encoding) → thousands of sparse binary features.
Each sample has mostly zeros, only a few 1s.
Observation:
Many features are *mutually exclusive* — they never take nonzero values simultaneously.
Example:

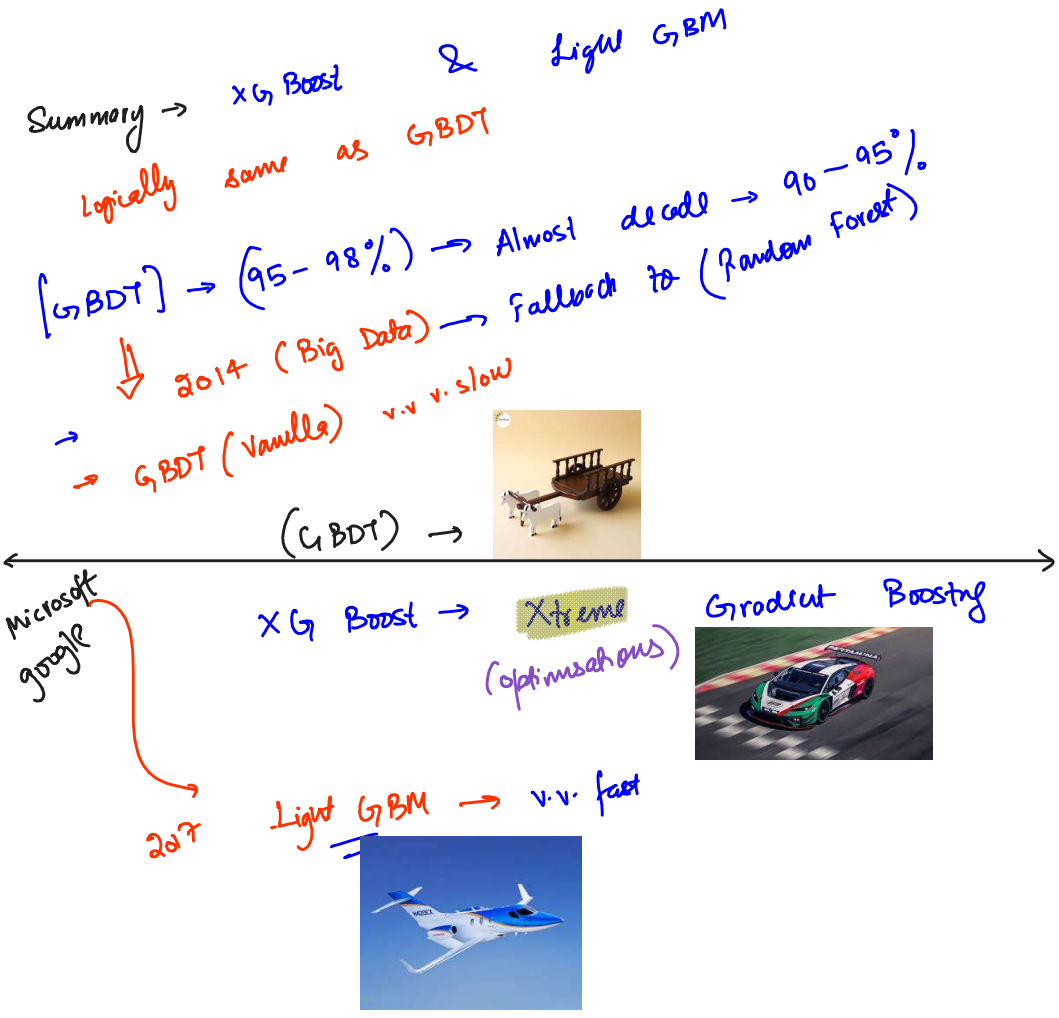
- gender_male and gender_female → one is 1, the other is 0.
- clicked_ad_A, clicked_ad_B, clicked_ad_C → user clicked only one ad.

If features never overlap, we can *bundle* them into a single numeric feature.
1. What EFB does
LightGBM scans all sparse features and finds groups that are *exclusive* (non-overlapping).
It then combines them into one dense feature by **offset encoding** — adding a unique offset for each original feature so values don't collide.
2. Why it works

- Fewer total features → smaller histograms → less memory & faster split-finding.
- No information loss, because exclusivity guarantees no overlap.

Think of EFB as merging multiple non-overlapping train tracks into a single rail line by scheduling trains at different times.
They never collide, but you save infrastructure.





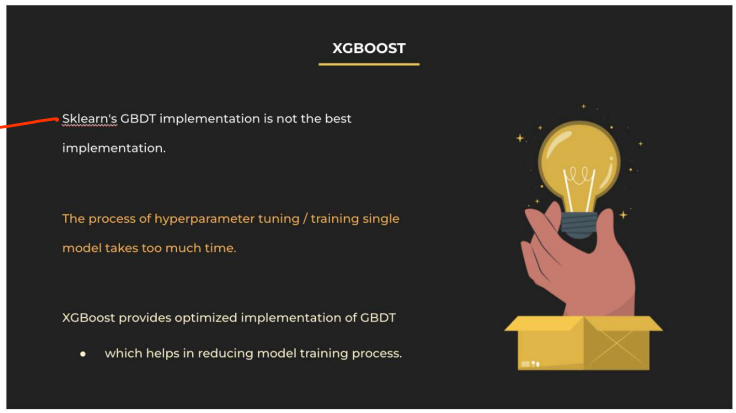
(*) How these are optimised

[1 to 1000] → C → 1 mnts

→ Python → 4 mnts

Understand How they are so fast

[Data Engineer + AI Architect]



Vanilla GBDT → slow

what optimisation XG Boost provide ?

[Decision Tree]

x_1 - - -	x_d	Y

Q How to select root Node ?

* (Information Gain)

* O(N x d * time to find IG)

(1 Billion) → (1B. Info gain)

Q Can you do better ? Yes

→ Parallelisn of feature selection

What optimization does XGBoost provides ?

1. Parallelization of features selection

While building Decision Trees,

- there will be n features to consider while splitting the node.

$f_1\ f_2\ f_3 \dots f_m$ — M features to consider for split

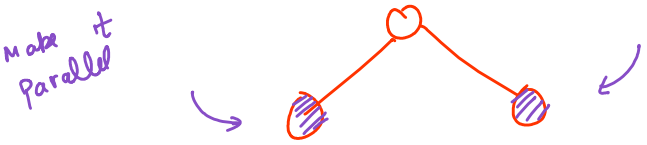


The computation of Information Gain(s) of the features is done in parallel

- which helps in reducing the training time

$f_1 : IG_1$
$f_2 : IG_2$
$\vdots$
$f_m : IG_m$

Compute in parallel

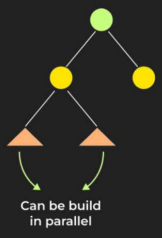


2. Parallelization in building DT

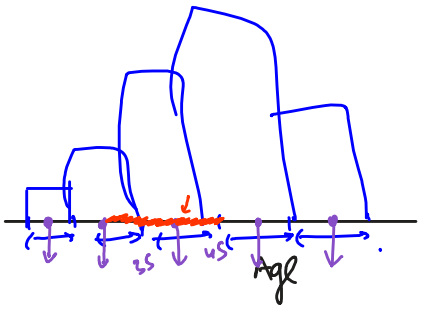
While building a DT,

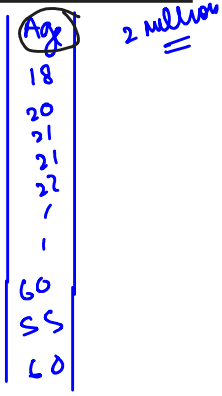
- both subtrees (left and right) can be build in parallel
- as there is no dependency between them.

which helps in making the process faster and efficient.



Q Can you optimise further ?



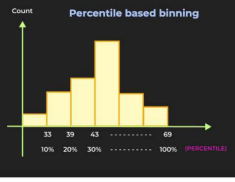


3. Optimizing thresholding in numerical feature

In conventional DTs,

- While finding the threshold for numerical feature
  - all the numerical values are tested to find one with maximum information gain.

SLOW



XGBoost optimizes this by using histogram based binning

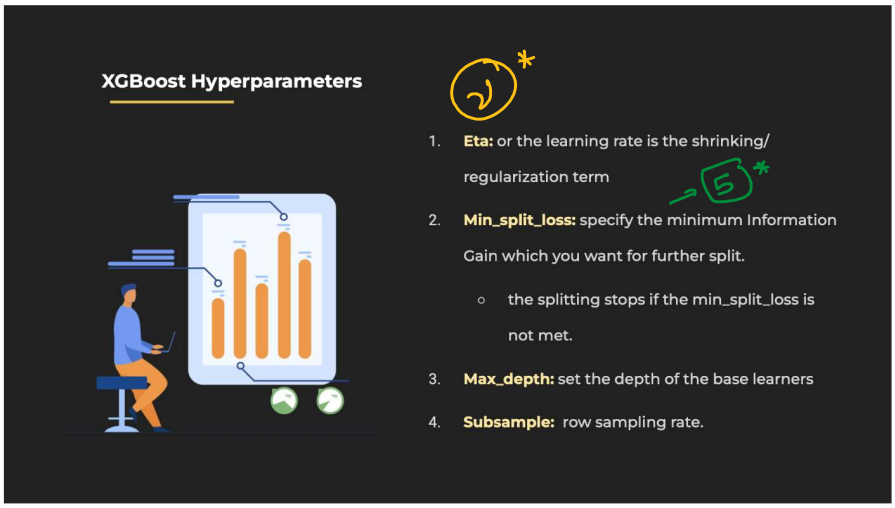
- it creates discrete bins (percentile binning) using these continuous values
- then selects threshold using the bins instead of trying every single value.

(XGBoost) (2015 -16) → Tabular → Hero

(U.W)

GBDT → (overfit) → (xGBoost) → (Regularisation)

Hyperparamters




```
Parameters:{'subsample': 0.6, 'n_estimators': 200, 'max_leaf_nodes': 80, 'max_depth': 5,
'learning_rate': 0.3, 'colsample_bytree': 0.6} Mean_score: 0.958523592085236 Rank: 4
Parameters:{'subsample': 0.8, 'n_estimators': 150, 'max_leaf_nodes': 40, 'max_depth': 7,
'learning_rate': 0.1, 'colsample_bytree': 0.8} Mean_score: 0.9615677321156774 Rank: 2
Parameters:{'subsample': 1.0, 'n_estimators': 200, 'max_leaf_nodes': 80, 'max_depth': 7,
'learning_rate': 0.2, 'colsample_bytree': 0.8} Mean_score: 0.9650558092338914 Rank: 1
Parameters:{'subsample': 0.8, 'n_estimators': 150, 'max_leaf_nodes': 40, 'max_depth': 5,
'learning_rate': 0.1, 'colsample_bytree': 0.8} Mean_score: 0.9540208016235413 Rank: 6
Parameters:{'subsample': 0.8, 'n_estimators': 100, 'max_leaf_nodes': 40, 'max_depth': 5,
'learning_rate': 0.2, 'colsample_bytree': 0.6} Mean_score: 0.9534500253678336 Rank: 7
Parameters:{'subsample': 0.6, 'n_estimators': 150, 'max_leaf_nodes': 20, 'max_depth': 5,
'learning_rate': 0.2, 'colsample_bytree': 0.8} Mean_score: 0.9607432775240993 Rank: 3
Parameters:{'subsample': 1.0, 'n_estimators': 50, 'max_leaf_nodes': 40, 'max_depth': 7,
'learning_rate': 0.1, 'colsample_bytree': 1.0} Mean_score: 0.9384830035514966 Rank: 9
Parameters:{'subsample': 0.6, 'n_estimators': 100, 'max_leaf_nodes': 40, 'max_depth': 7,
'learning_rate': 0.2, 'colsample_bytree': 0.6} Mean_score: 0.9571283612379503 Rank: 5
Parameters:{'subsample': 1.0, 'n_estimators': 200, 'max_leaf_nodes': 80, 'max_depth': 4,
'learning_rate': 0.1, 'colsample_bytree': 0.6} Mean_score: 0.9431760527650939 Rank: 8
Parameters:{'subsample': 0.8, 'n_estimators': 50, 'max_leaf_nodes': 40, 'max_depth': 3,
'learning_rate': 0.3, 'colsample_bytree': 0.6} Mean_score: 0.9205986808726534 Rank: 10
```

Most app

What are the parameters used to control overfitting in Tree based model?

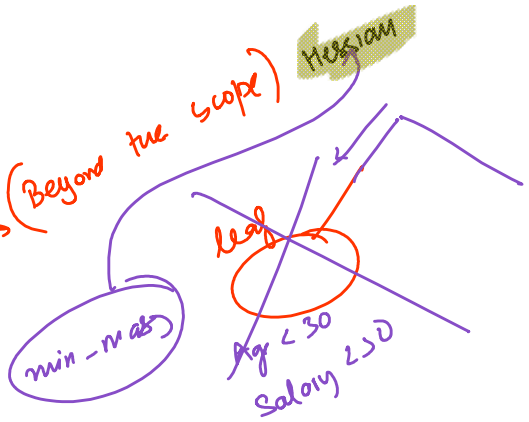
4 options

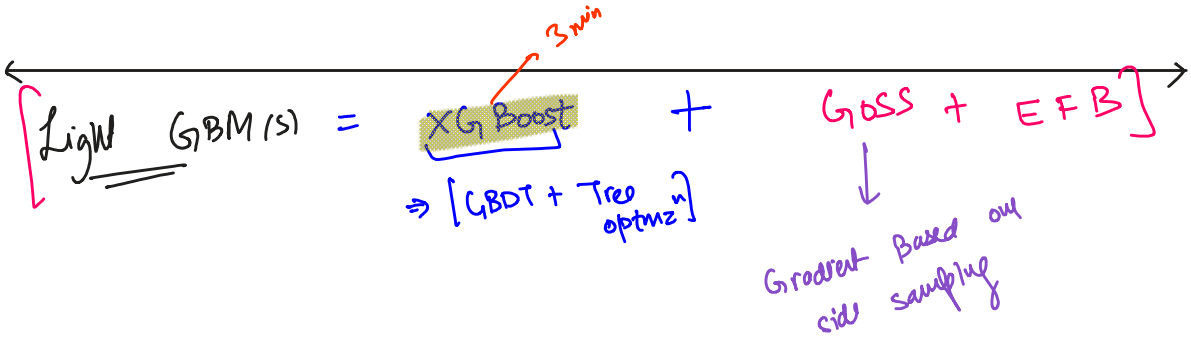
Active Duration (Most preferred: 30 seconds)

Appears for 30 Secs

- A max_depth, subsample
- B min_child_weight, max_depth
- C max_depth, n_estimator
- D None of the above

(Beyond the scope)





What makes LightGBM faster ?

1. **GOSS (Gradient-based One-Side Sampling)**

Say, we are training the mth model,

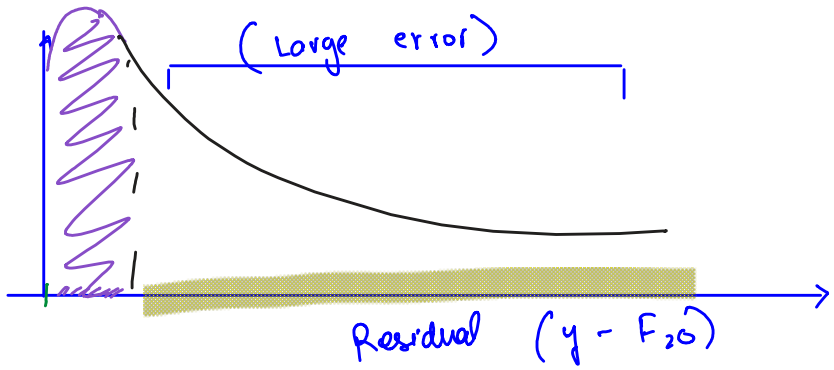
- During training, there will be lot of data points with small residual

LightGBM will drop all the points from training where the error is very small.

Intuitively,

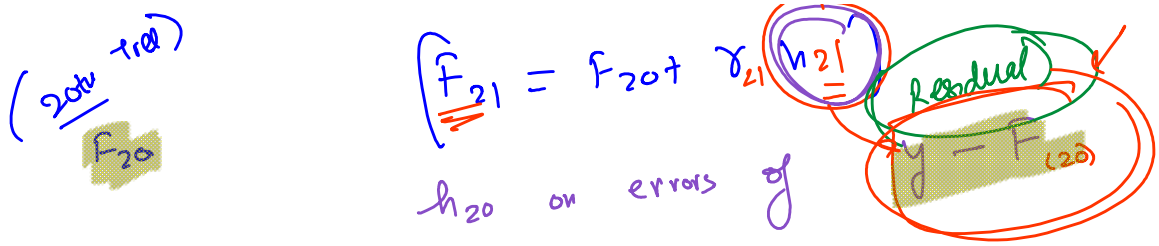
- It is doing **smart sampling** by
  - reducing the size of training data
  - which make the training process faster.





(Last tree)

$F_{21} = F_{20} + \gamma_{21} h_{21}$ (residual)



22:20

What makes LightGBM faster ?

1. **GOSS** (Gradient-based One-Side Sampling)

Say, we are training the mth model,

- During training, there will be lot of data points with small residual

LightGBM will drop all the points from training where the error is very small.

Intuitively,

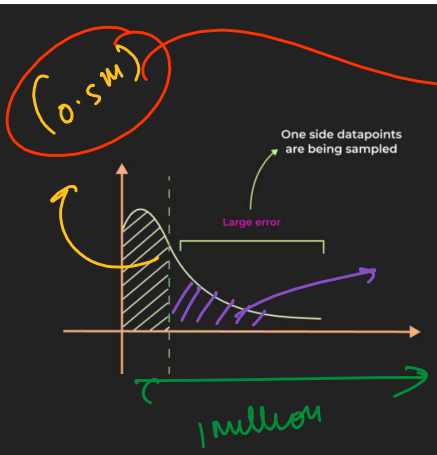
- It is doing **smart sampling** by
  - reducing the size of training data
  - which make the training process faster.



Why does one side sampling means ?

If we were to plot the error distribution,

- We are sampling the points from one side of the distribution



(0.5M)

0.5M + 0.05M

0.55M

h_{20}

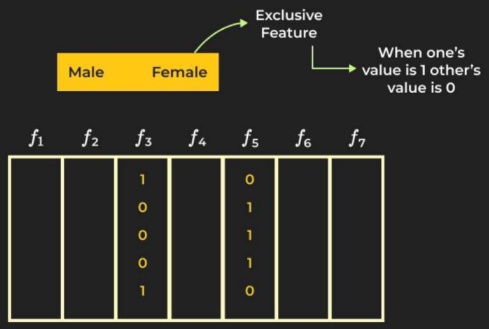
(2) EFB → exclusive feature Bundling

→ To decrease the dimensionality of your data

2. EFB (Exclusive Feature Bundling)

It scans through all the features

- tries to find feature pairs which are exclusive

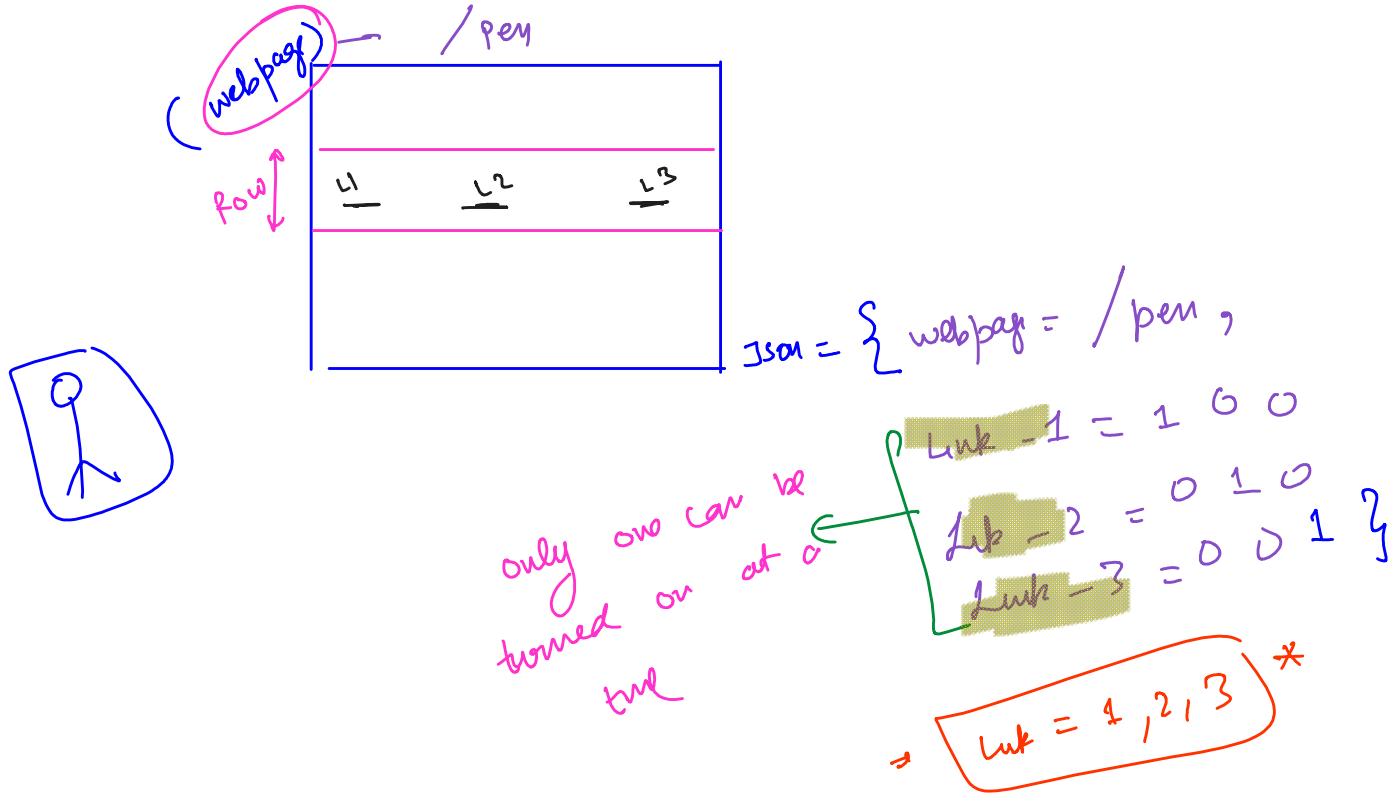


What does exclusive feature mean ?

Say, we have an OHE encoded categorical feature

For example:

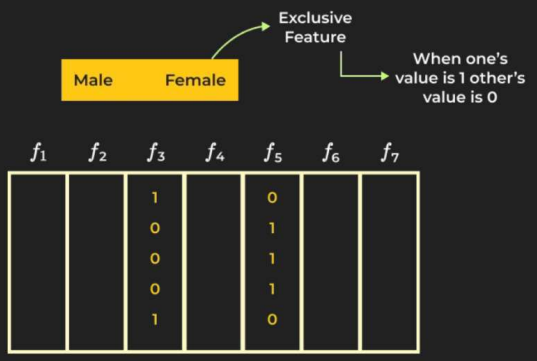
- **Male & Female.**



2. EFB (Exclusive Feature Bundling)

It scans through all the features

- tries to find feature pairs which are exclusive

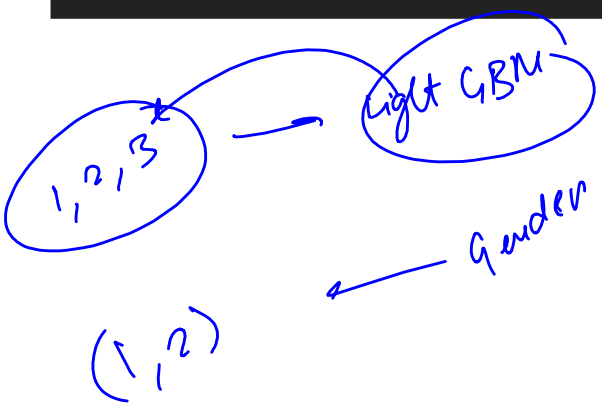


What does exclusive feature mean ?

Say, we have an OHE encoded categorical feature

For example:

- **Male & Female.**



---

* (Stacking) → (Not used frequently)

→ Stack of ML models

Stacking

(1 file)

How does stacking works ?

Let's say we are entering a kaggle competition

- **and we have a team of m members** (20)



The team decided that each individual member will train its own model.

So, on a give training dataset - there will be m well hyperparameter tuned model

Note that

- all m models can be different
- i.e. C_1 can be logistic regression, C_2 can be Knn etc

(m models)

(20 = m models)

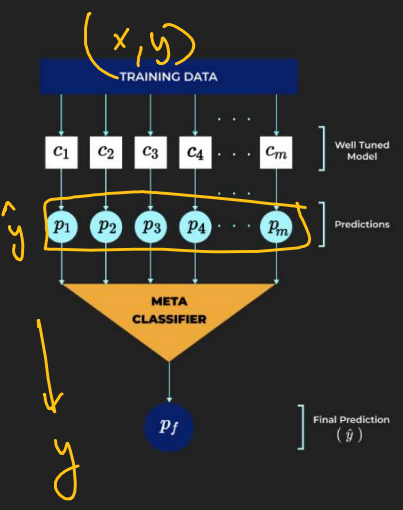
Train on final model → Meta classifier (simple model)

How do we combine them?

Each model will give us a prediction (P_1, P_2, ... P_n)

- We train another model (**Meta Classifier**)
  - using predictions as input data
  - and original target variable (y) as target variable

Note : Instead of predictions, we can use class probabilities as input feature for meta classifier



The prediction given by Meta Classifier is treated as final prediction

What is stacking?

4 options

Active Duration(Most preferred: 30 seconds)

Appears for 30 Secs

A. A meta-algorithm that combines the predictions of multiple ensemble models

B. A method for reducing the dimensionality of the feature space

C. A method for adjusting the weights of different models in an ensemble

D. A method for combining the predictions of different models in an ensemble

why don't we use it ?

- **extensive time complexity**

It takes a lot of time to fine tune M base models

- and then training the predictions on Meta classifier

NOTE :

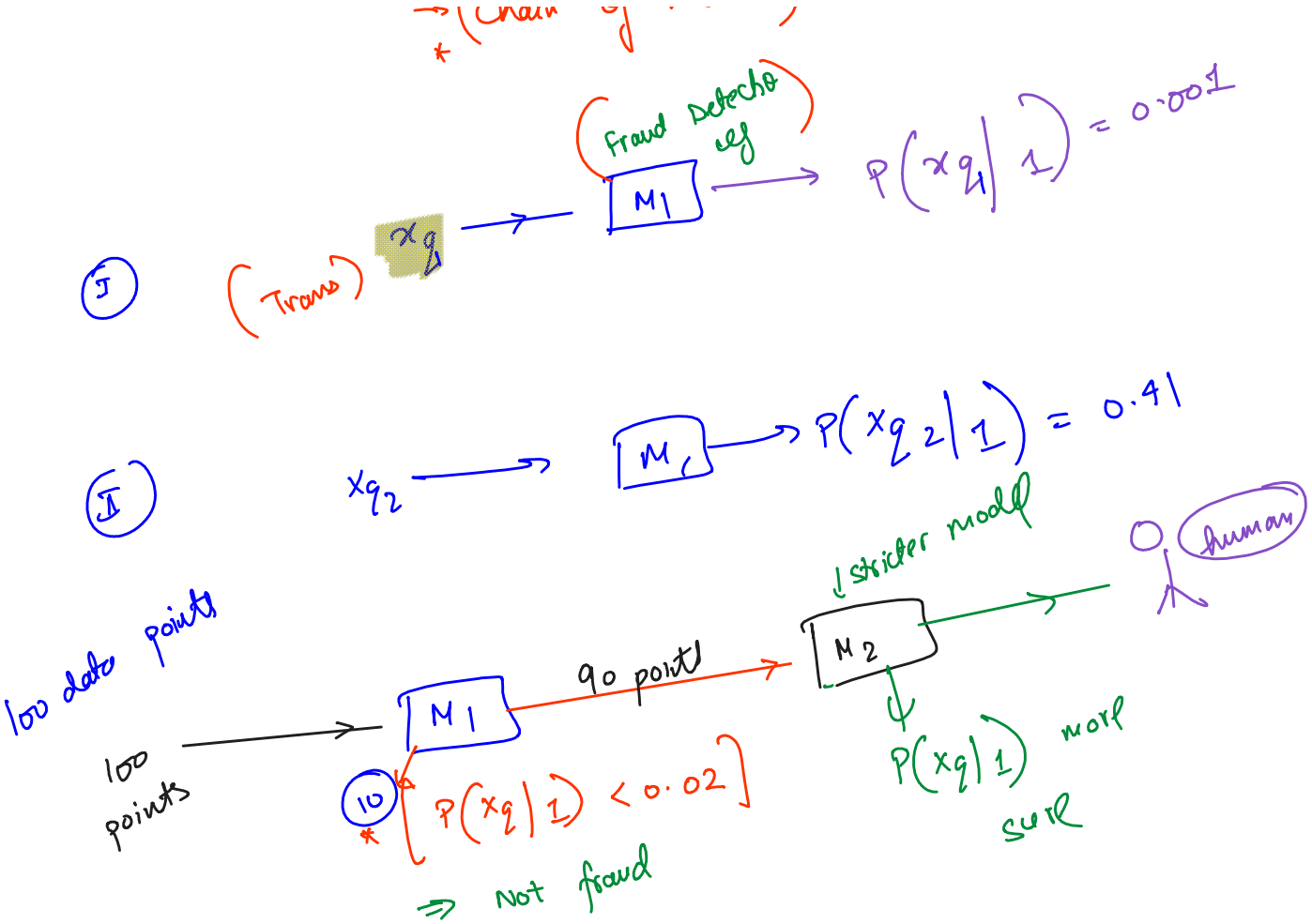
- Stacking is extensively used in kaggle competition as kaggle competition goal is to get the top score
- and not the fast models



Cascading *

→ (chain of models)

*



CASCADING

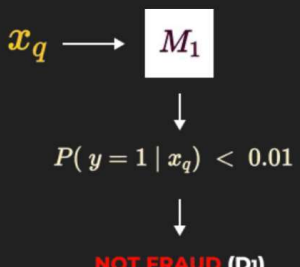
Chaining of models

Let D = dataset [transaction fraud or not]

- Y = 1 (fraud)
- Y = 0 (not fraud)

For a query point

- we will pass this point through the first model M_1
- Model M_1 will return the probability of the query point being a fraud





Based on probability, we'll split it in 2 parts:

- if the probability of $\hat{y}$ being 1 is extremely low, say < 0.01 then
  - we consider that as not fraudulent, let this data be D_1

What happens to rest of the data?

The rest of the points ($D - D_1$) i.e. data with prob. > 0.01 which we are not sure about

- will be passed through the next model M_2
- Model M_2 will be more stricter i.e. it'll penalize more.



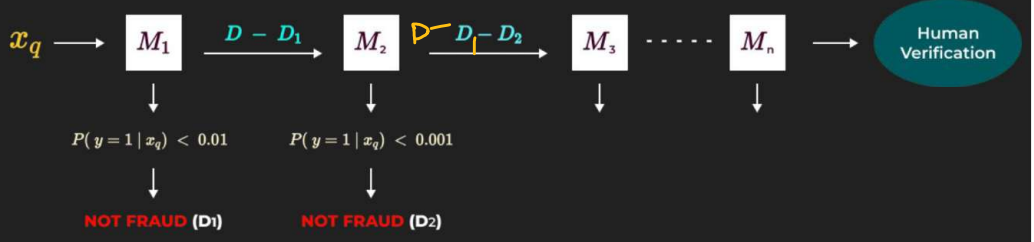
Again model M_2 will split into 2 parts

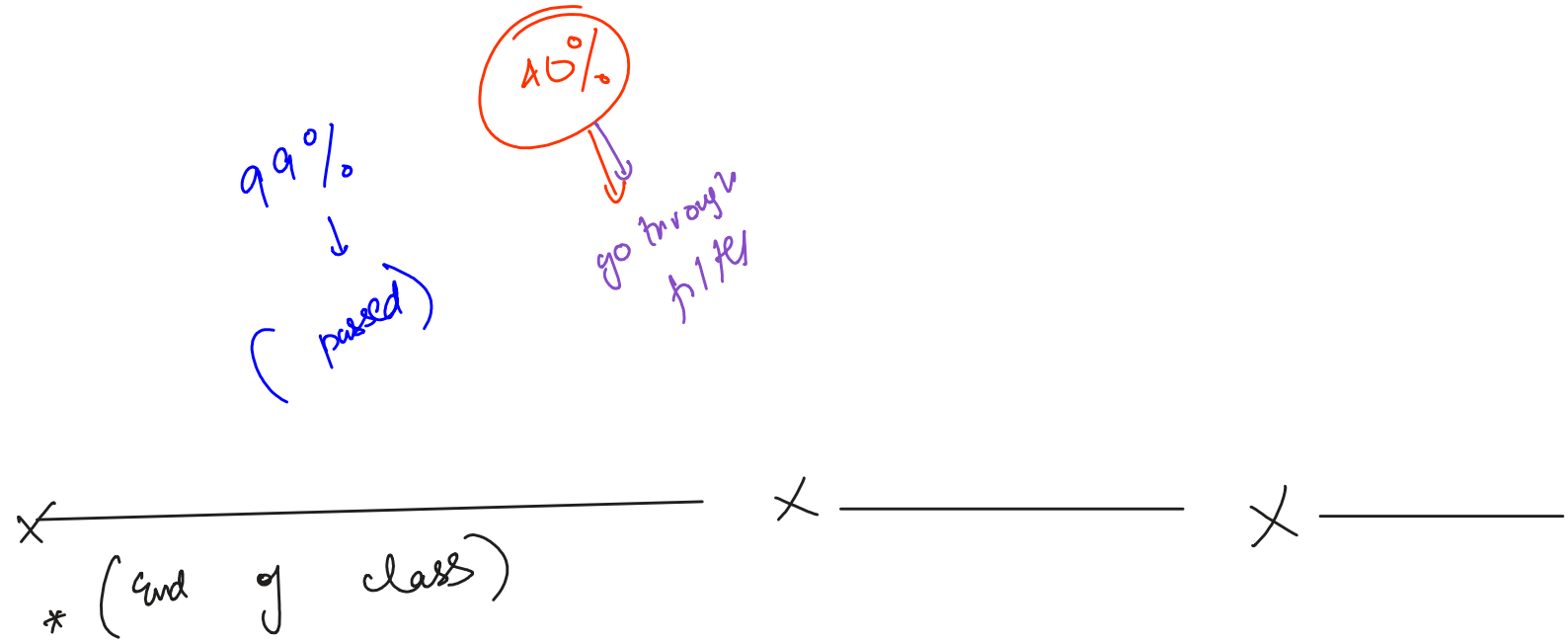
- non fraud (say, $P(y=1|x_q)<0.001$)
- fraud transac. ($p > 0.001$)

We keep doing this till n models

* (20 models)

Training Data : D




99%
(passed)
40%
go through file
* (end of class)